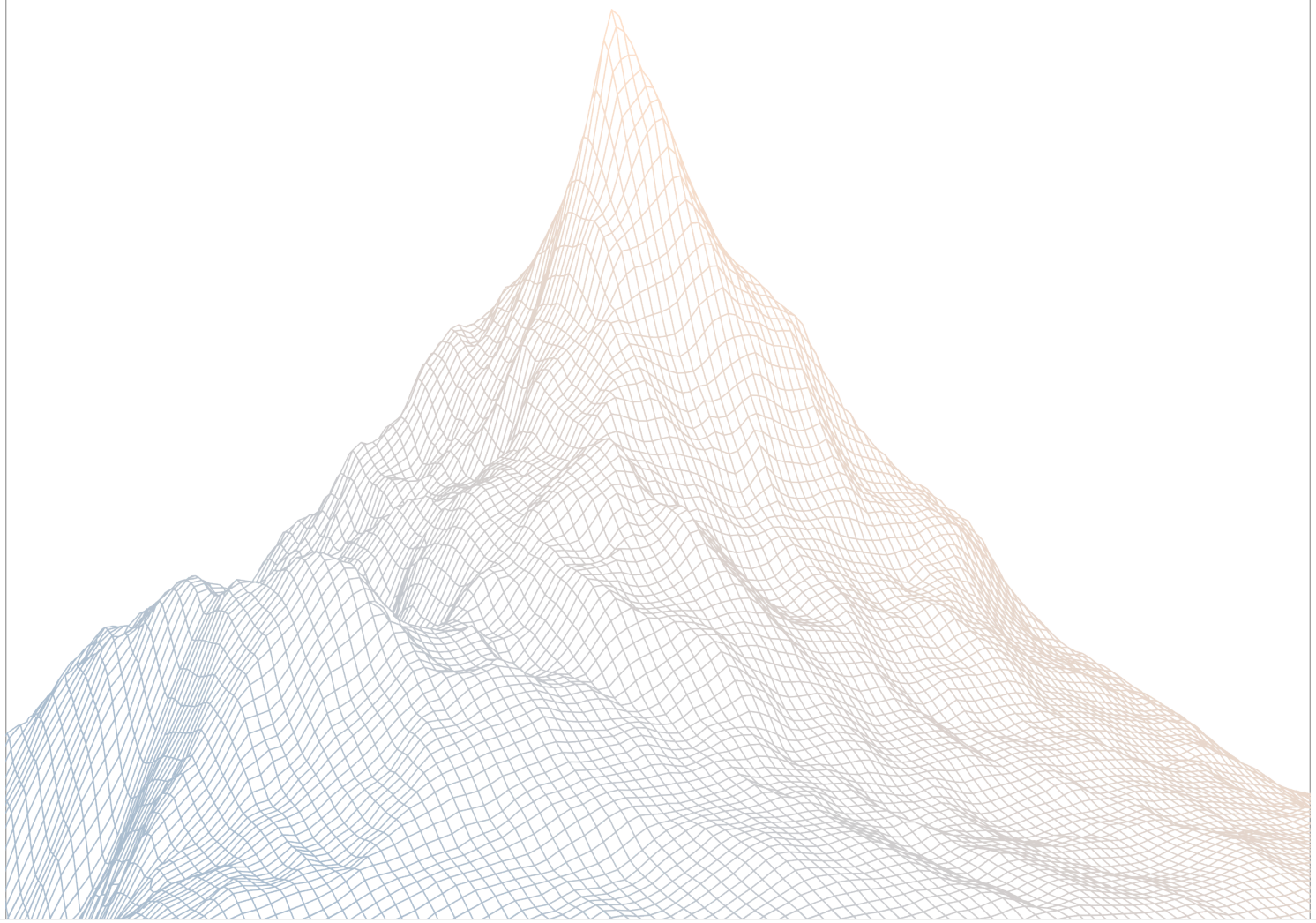
Zenith

D3

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: September 9nd to September 16th, 2025

AUDITED BY: IIIIIII
peakbolt

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About D3 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 7

4.1 Critical Risk 8

4.2 High Risk 13

4.3 Medium Risk 15

4.4 Low Risk 21

4.5 Informational 33

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2 Executive Summary

2.1 About D3

Solana Record Service (SRS) is your gateway to opening up the identity layer of .sol domain names which allows you to build the bridge between your Web2 and Web3 identities.

2.2 Scope

The engagement involved a review of the following targets:

Target	solana-record-service
Repository	https://github.com/solana-foundation/solana-record-service
Commit Hash	`d98ba1eb6a09de1b6cf8f2bc6e3b6418f5886e25`
Files	`program/src/**.rs`

2.3 Audit Timeline

September 9th, 2025	Audit start
September 16th, 2025	Audit end
September 16th, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	3
High Risk	1
Medium Risk	6
Low Risk	12
Informational	8
Total Issues	**30**

3

Findings Summary

ID	Description	Status
C-1	check_owner_or_delegate_tokenized() fails to validate mint of token account	Open
C-2	Missing validation of delegate's class in validate_delegate()	Open
C-3	Permissioned class access control can be bypassed with MintTokenizedRecord	Open
H-1	UpdateRecord fails to check record is not frozen	Open
M-1	Records frozen prior to being minted become unfrozen once burned	Open
M-2	No method of changing a class' authority	Open
M-3	Record tokenization can be DOSed	Open
M-4	Record tokenization loses the original record ownership	Open
M-5	PDA creation can be blocked by lamport donations	Open
M-6	Lack of role separation for freezing of records	Open
L-1	Long gTLDs are not supported	Open
L-2	Records tokenizations over-pay their mint account rent	Open
L-3	Account resizes do not take into account the MAX_PERMITTED_DATA_INCREASE limitation	Open
L-4	Code does not recognize deployment key changes	Open
L-5	Metadata not validated during record tokenization	Open
L-6	Unused expiry state variable in Record struct	Open
L-7	Record base metadata size is over-constrained	Open
L-8	Record tokenization may fail with full-sized additional metadata fields	Open

ID	Description	Status
L-9	Generic InitializeGroup and InitializeMetadata CPIs may break with future uses	Open
L-10	Token group membership counts double count re-tokenized records	Open
L-11	Token group membership counts can be made to be incorrect	Open
L-12	Wrong rent may be paid if rent exemption requirements change	Open
I-1	Dead code	Open
I-2	Misleading comments	Open
I-3	Typos	Open
I-4	burn_tokenized_record() should close the owner token account	Open
I-5	burn_tokenized_record() might not refund rent to the original payer	Open
I-6	unnecessary refund code in DeleteRecord::execute()	Open
I-7	Semantics of Mint::check_initialized() are unclear	Open
I-8	CU optimizations	Open

4

Findings

4.1 Critical Risk

A total of 3 critical risk findings were identified.

[C-1] `check_owner_or_delegate_tokenized()` fails to validate mint of token account

SEVERITY: Critical	IMPACT: High
STATUS: Open	LIKELIHOOD: High

Target

- record.rs#L177-L233

Description:

`check_owner_or_delegate_tokenized()` will check that the authority is the owner of the token account.

However, it fails to check that the token account's mint matches the record's mint (record owner).

This allows an attacker to freeze/transfer any tokenized records by being a token account owner. The attacker can also burn the tokenized record to take over as the record owner.

Recommendations:

Update `check_owner_or_delegate_tokenized()` to verify that the token account's mint matches the record's mint (record owner).

D3:

Zenith:

[C-2] Missing validation of delegate's class in `validate_delegate()`

SEVERITY: Critical	IMPACT: High
STATUS: Open	LIKELIHOOD: High

Target

- record.rs#L124-L140

Description:

`validate_delegate()` fails to check that the delegate's class matches record's class.

That means an attacker can modify records in any class by becoming a permission class authority. This include delete, transfer and mint tokenized record.

```
pub fn validate_delegate(
    class: &AccountInfo,
    authority: &AccountInfo,
) → Result<(), ProgramError> {
    Class::check_program_id(class)?;

    let class_data = class.try_borrow_data()?;

    if class_data[IS_PERMISSIONED_OFFSET].ne(&1u8) {
        return Err(ProgramError::InvalidAccountData);
    }

    unsafe {
        Class::check_discriminator_unchecked(&class_data)?;
        Class::check_authority_unchecked(&class_data, authority)
    }
}
```

Recommendations:

In `validate_delegate`, check that delegate's class matches record's class.

D3:

Zenith:

[C-3] Permissioned class access control can be bypassed with `MintTokenizedRecord`

SEVERITY: Critical	IMPACT: High
STATUS: Open	LIKELIHOOD: High

Target

- record.rs#L163

Description:

The `MintTokenizedRecord` instruction will call `Record::check_owner_or_delegate()` to validate the record and the class to ensure they are validate PDA with the correct program id and discriminator.

However, `Record::check_owner_or_delegate()` will actually do a early return and skip the class validation when the authority is the owner.

That means a record owner can pass in any arbitrary account for the class parameter to mint a tokenized record.

This allows an attacker as a record owner, to mint any tokenized record and initialize it under a different permissioned class's group mint. This effectively bypass the permissioned class access control, as the attacker can add any record token to that group mint.

Impact is rated High for this issue as it impacted data integrity, which is important in this case, despite no financial loss.

```
pub fn check_owner_or_delegate(
    record: &AccountInfo,
    class: Option<&AccountInfo>,
    authority: &AccountInfo,
) → Result<(), ProgramError> {
    // Check the program id and the discriminator
    Self::check_program_id_and_discriminator(record)?;

    // Check if the authority is signer
    if !authority.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }
```

```
    let data = record.try_borrow_data()?;

    // Check if the authority is the owner
    if authority
        .key()
        .eq(&data[OWNER_OFFSET..OWNER_OFFSET + size_of::<Pubkey>()])
    {
        return Ok(());
    }

    // Check if the owner type is pubkey
    if data[OWNER_TYPE_OFFSET].ne(&(OwnerType::Pubkey as u8)) {
        return Err(ProgramError::InvalidAccountData);
    }

    // Validate the delegate
    let class = class.ok_or(ProgramError::MissingRequiredSignature)?;
    Self::validate_delegate(class, authority)
}
```

Recommendations:

For `MintTokenizedRecord`, validate the class account even when the authority is the record owner.

D3:

Zenith:

4.2 High Risk

A total of 1 high risk findings were identified.

[H-1] `UpdateRecord` fails to check record is not frozen

SEVERITY: High	IMPACT: High
STATUS: Open	LIKELIHOOD: Medium

Target

- update_record.rs#L72-L85

Description:

When a record is frozen, it should not allow any update of the record.

However, `UpdateRecord` fails to check record is not frozen, and allows one to update the record.

```
impl<'info> UpdateRecord<'info> {
    pub fn process(ctx: Context<'info>) -> ProgramResult {
        #[cfg(not(feature = "perf"))]
        sol_log("Update Record");
        Self::try_from(ctx)?.execute()
    }

    pub fn execute(&self) -> ProgramResult {
        // Update the record data [this is safe, check safety docs]
        unsafe {
            Record::update_data_unchecked(self.accounts.record,
    self.accounts.payer, self.data)
        }
    }
}
```

Recommendations:

Modify `UpdateRecord` to check that the record is not frozen before allowing record update.

D3:

Zenith:

4.3 Medium Risk

A total of 6 medium risk findings were identified.

[M-1] Records frozen prior to being minted become unfrozen once burned

SEVERITY: Medium	IMPACT: High
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/burn_tokenized_record.rs

Description:

The project allows records to be frozen, which prevents them from being updated and transferred. The documentation mentions:

> Domain freezing capability must also be implemented to temporarily prevent transfers of domains when required for legal or compliance reasons. For records, this is simply allowed by allowing for a freeze authority to prevent updates.

If a record were frozen prior to being tokenized, once the record is tokenized then burned, the record will be automatically unfrozen, allowing illegal/non-compliant modifications of the record.

Recommendations:

Do not call `update_is_frozen_unchecked()` during mint or burn, and instead reject modification instructions when the record has its `owner_type` set to `OwnerType::Token`

D3:

Zenith:

[M-2] No method of changing a class' authority

SEVERITY: Medium	IMPACT: High
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/state/class.rs

Description:

There is no instruction for modifying a class' `authority` field. This means that in the case of a key compromise, or in cases where the class has key rotation requirements, it's not possible to change the class' authority. In the case of key compromise of a permissioned class, this means that an attacker will be able to wreak havoc on all of the class' associated records.

Recommendations:

Add a modification instruction, and use a nonce instead of the authority in each class' seed

D3:

Zenith:

[M-3] Record tokenization can be DOSed

SEVERITY: Medium	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Medium

Target

- program/src/instructions/mint_tokenized_record.rs

Description:

Each class is supposed to represent a permissioned container for all of the records and sub-records it knows about. One documented use case is that of having one ICANN-compliant gTLD per class. One such gTLD is that of `.xyz` which has millions of domains, which would require at least as many records to be fully stored on-chain. In order to facilitate the enumeration of each class' tokenized records, each tokenized record is added as a token group member to a token group for the class. However, when the group is created, the member size is limited to only 100 entries. This means that once 100 `.xyz` domains/sub-domains have been tokenized, no others can be tokenized until some of the existing ones have been burned. To accomplish a DOS of domain tokenization, an attack need only buy a single domain, create 100 sub-domains, then tokenize them all.

Note that this failure is only evident once a gTLD has had 100 domains/sub-domains tokenized, which likely will be well after many more non-tokenized records have been created for that class. Once a record is created for a class, there is no way to change classes, so to work around the DOS, each domain will have to be binned re-created in gTLD-specific-binned class.

Recommendations:

Create `emit_cpi!()`-like functionality in pinocchio and use that to track changes off-chain, rather than using token groups

D3:

Zenith:

[M-4] Record tokenization loses the original record ownership

SEVERITY: Medium	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Medium

Target

- program/src/instructions/mint_tokenized_record.rs
- program/src/instructions/burn_tokenized_record.rs

Description:

Record freezes by a permissioned class can be trivially undone by the record owner via a supported instruction. The project documentation refers to the intention of having the ability to have the ownership of a domain separate from the authority of the record, and potentially having a web interfaces that acts as an intermediary which allows the domain owner to get the required record authorization signature. In such a setup, if the record is ever tokenized and given to the user, the user can bypass all record authorization requirements imposed by the web interface, by burning the record's token. The burn instruction, rather than assigning ownership to the original authority, gives it to the owner of the token account that holds the token. One the token is burned, the user can make the desired changes in the record, then re-tokenize the record.

Recommendations:

Don't over-write the original owner and, if the class is permissioned, only allow the original owner to burn or freeze/unfreeze the token

D3:

Zenith:

[M-5] PDA creation can be blocked by lamport donations

SEVERITY: Medium	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Medium

Target

- create_class.rs#L161
- create_record.rs#L148
- mint_tokenized_record.rs#L216
- mint_tokenized_record.rs#L293

Description:

The program uses PDAs for the class, record, group and mint accounts.

However, it uses calls the `create_account` instruction to create the PDA, and this allows one to DoS the PDA creation by donating lamports to the PDA.

When the PDA has a non-zero lamport balance, the `create_account` instruction will fail due to this check.

This issue will allows an attacker to DoS creation of class and record, and minting of tokenized record.

Recommendations:

This can be resolved by splitting into separate allocate, transfer, and assign instructions.

D3:

Zenith:

[M-6] Lack of role separation for freezing of records

SEVERITY: Medium	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Medium

Target

- freeze_record.rs
- freeze_tokenized_record.rs

Description:

The records can be frozen by the owners or delegates, to prevent update of the records by themselves.

However, that means the owners/delegates can just unfreeze the records to perform the update as they also have the permission to unfreeze the record.

As owners/delegates have both update and freeze permissions, it makes the freeze functionality useless.

Recommendations:

Add a separate freeze_authority for the record, to allow a separate signer to control the freeze/unfreeze of records.

The same applies for tokenized records.

D3:

Zenith:

4.4 Low Risk

A total of 12 low risk findings were identified.

[L-1] Long gTLDs are not supported

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/create_class.rs
- program/src/constants.rs

Description:

The `name` field of each class, when the class is for a gTLD, is meant to be the gTLD itself. The code, when performance is disabled, limits class names to 32 characters, but this is much smaller than the maximum specified by ICANN:

> General use TLDs may now be longer than two or three characters in length. In fact, TLDs can be anywhere from two to 63 characters long.

https://www.icann.org/en/system/files/files/ua-factsheet-a4-17dec15-en.pdf

Recommendations:

Introduce and use a `MAX_CLASS_NAME_LEN`, and set it to 255, which is the maximum supported by `write_str_with_length()`

D3:

Zenith:

[L-2] Records tokenizations over-pay their mint account rent

SEVERITY: Low	IMPACT: Low
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/mint_tokenized_record.rs
- program/src/state/record.rs

Description:

For every token extension, the account creator has to include, among other things, the size of the extension type (2 bytes), the size of the length of the extension data (2 bytes), and the extension data itself. During the creation of the mint account, the code uses constants (with correct values) for all of the static-sized extensions, but when it comes to the token metadata extension, it relies on the `Record::get_metadata_len_unchecked` function to determine the right length. This function incorrectly includes part of the size of the `Record` struct (`SEED_LEN_OFFSET`, plus the size of the `seed` array length, plus the actual seed length) in the offset that is ultimately returned as the size. When the seed array length is zero, the size returns happens to match the required token metadata extension tlv, but for every byte the seed array is larger than that, the size is also increased, and rent is charged on this excess as well.

Recommendations:

Modify the `get_metadata_len_unchecked()` function to only return the length of the actual metadata, and use a constant in the `create_mint_account()` function to account for the type, size, update_authority, and mint. The `TOKEN_2022_MEMBER_LEN` constant also needs to be updated to account for the `member_number` field of size 8.

D3:

Zenith:

[L-3] Account resizes do not take into account the `MAX_PERMITTED_DATA_INCREASE` limitation

`SEVERITY:` Low	`IMPACT:` Low
`STATUS:` Open	`LIKELIHOOD:` Low

Target

- program/src/instructions/update_record.rs
- program/src/utils.rs

Description:

A top-level instruction can add at most 10,024 bytes to an account using `realloc()`, but there are no validations done during instruction processing nor in the `resize_account()` function itself to ensure that changes to metadata stay under this value, despite the unused `MAX_RECORD_SIZE` having a value of 1MB.

Recommendations:

Explicitly reject update requests during the instruction parsing logic if they're too large

D3:

Zenith:

[L-4] Code does not recognize deployment key changes

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/lib.rs

Description:

The program hard-codes an array of bytes as the crate's `Pubkey`, but if the deployment key has to change prior to deployment, there is nothing that will catch that the `crate::ID` doesn't match, until it's time to use PDAs, at which point validations will fail.

Recommendations:

Check that the `_program_id == crate::ID` in `process_instruction()` or pass along the program entrypoint's program id, in the `utils::Context`, and use that rather than the hard-coded ID.

D3:

Zenith:

[L-5] Metadata not validated during record tokenization

SEVERITY: Low	IMPACT: Low
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/mint_tokenized_record.rs

Description:

During record tokenization, the `Record.data` field is assumed to contain data in the format expected by the token metadata extension. There, however, are no checks that this is the case, so records with arbitrary bytes may cause the parsing code to attempt to read past the end of the account buffer, causing the instruction to fail cryptically.

Recommendations:

Fail with a separate error code if the lengths read will cause a read past the end of the account data

D3:

Zenith:

[L-6] Unused `expiry` state variable in `Record` struct

SEVERITY: Low	IMPACT: Low
STATUS: Open	LIKELIHOOD: Low

Target

- record.rs#L37-L38

Description:

The `Record` has an `expiry` variable that is not used for any purpose on-chain, such as preventing update/transfer when the record has expired.

```
pub struct Record<'info> {
    /// The class this record belongs to
    pub class: Pubkey,
    /// The owner_type enum
    pub owner_type: OwnerType,
    /// The owner of this record
    pub owner: Pubkey,
    /// Whether the record is frozen
    pub is_frozen: bool,
    /// Optional expiration timestamp, if not set, the expiry is [0; 8]
    pub expiry: i64,
    /// The record name/key
    pub seed: &'info [u8],
    /// The record's data content
    pub data: &'info str,
}
```

Recommendations:

Consider removing the `expiry` if it is not needed anymore. Otherwise implement the necessary functions for expiry or clarify the purpose of the expiry field in the documentation.

D3:

Zenith:

[L-7] Record base metadata size is over-constrained

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/token2022/initialize_metadata.rs

Description:

The maximum size of record base metadata is limited to 255 bytes. The base metadata is meant to include the name (likely includes the domain label whose maximum is 63 bytes), the symbol (also likely 63 bytes), the URI (gTLDs have a maximum of 68), and the lengths of those fields (4 * 3). This leaves only 49 bytes for the rest of the length of the URI (the path/file), which may not be large enough for all cases.

Recommendations:

Increase the `instruction_data`'s size to allow for `MAX_METADATA_LEN` bytes for _each_ field, rather than all of them together.

D3:

Zenith:

[L-8] Record tokenization may fail with full-sized additional metadata fields

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/token2022/update_metadata.rs

Description:

The CPI builder for updating additional metadata fields does not properly include the size of the field identifier, which means there is one fewer bytes available than the `MAX_METADATA_LEN` maximum. Since the length of the field's value is included in the instruction, the CPI will fail if the actual value gets truncated when the size of the key and value (and the size of their lengths) is exactly equal to the `MAX_METADATA_LEN`.

Recommendations:

Include the size of the field identifier in the size of `instruction_data`

D3:

Zenith:

[L-9] Generic `InitializeGroup` and `InitializeMetadata` CPIs may break with future uses

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/token2022/initialize_group.rs

Description:

The use of the `InitializeGroup` CPI data currently passes the token group mint account as both the `mint` and `mint_authority` accounts, and the instruction only provides the `mint` account in the invocation's `account_infos` array. If a future use of this instruction passes `mint` and `mint_authority` accounts that are different from one another, the instruction will fail. The `InitializeMetadata` CPI has a similar issue in that the `update_authority` is not passed.

Recommendations:

Provide all accounts in the `account_infos` arrays, or update the structs to only take in a single account.

D3:

Zenith:

[L-10] Token group membership counts double count re-tokenized records

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/burn_tokenized_record.rs

Description:

During record tokenization the token mint for the record is added to the token group of the record's class but when the token is burned the token mint is not removed from the group. This means that when the record is re-tokenized, the mint will be added to the group again, increasing the member count again.

Recommendations:

Remove the member from the group prior to closing the mint account

D3:

Zenith:

[L-11] Token group membership counts can be made to be incorrect

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/mint_tokenized_record.rs

Description:

During record tokenization the token mint for the record is added to the token group of the record's class, which updates the token group's count of members. However, there is no validation that the group account provided to the tokenization instruction actually matches the class' token group account. A user can transfer the record to a PDA, create a token group owned by the PDA, then have the PDA sign the tokenization of the record. This would cause the PDA's token group member count to change rather than changing the class' token group member count.

Recommendations:

Validate that the `accounts.group` matches the class' group PDA

D3:

Zenith:

[L-12] Wrong rent may be paid if rent exemption requirements change

SEVERITY: Low	IMPACT: Medium
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/delete_record.rs
- program/src/constants.rs

Description:

During record deletion, the number of lamports in the account is reduced to be the hard-coded amount currently required to store one byte of account data (along with the account size itself). If the amount of rent required for rent exemption ever changes, records will either be unable to be closed, or will over-pay the amount of rent required.

Recommendations:

Use `Rent::get()?.minimum_balance(1)` instead

D3:

Zenith:

4.5 Informational

A total of 8 informational findings were identified.

[I-1] Dead code

SEVERITY: Informational	IMPACT: Informational
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/state/class.rs
- program/src/token2022/constants.rs
- program/src/utils.rs
- program/src/utils.rs
- program/src/constants.rs
- program/src/constants.rs
- program/src/state/record.rs
- program/src/state/record.rs
- program/src/state/record.rs
- program/src/token2022/burn.rs
- program/src/token2022/close_account.rs
- program/src/token2022/constants.rs
- program/src/token2022/constants.rs
- program/src/token2022/freeze_account.rs
- program/src/token2022/initialize_account_3.rs
- program/src/token2022/initialize_group.rs
- program/src/token2022/initialize_member.rs
- program/src/token2022/initialize_metadata.rs
- program/src/token2022/mint_to_checked.rs
- program/src/token2022/state.rs
- program/src/token2022/thaw_account.rs
- program/src/token2022/transfer_checked.rs
- program/src/token2022/update_metadata.rs

Description:

The targets listed above point to code that is never executed, which adds unnecessary cognitive load during reviews.

Recommendations:

Remove the unused code, or refactor/rewrite other code to call the functions/use the errors.

D3:

Zenith:

[I-2] Misleading comments

SEVERITY: Informational	IMPACT: Informational
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/create_class.rs
- program/src/instructions/delete_record.rs
- program/src/instructions/transfer_tokenized_record.rs
- program/src/utils.rs
- program/src/instructions/burn_tokenized_record.rs
- program/src/instructions/create_record.rs
- program/src/instructions/mint_tokenized_record.rs
- program/src/instructions/mint_tokenized_record.rs
- program/src/instructions/transfer_tokenized_record.rs
- program/src/token2022/initialize_member.rs
- program/src/token2022/initialize_metadata_pointer.rs
- program/src/token2022/initialize_permanent_delegate.rs
- program/src/token2022/transfer_checked.rs
- program/src/token2022/initialize_group.rs
- program/src/token2022/update_metadata.rs
- program/src/utils.rs

Description:

The following comments are misleading, which adds unnecessary cognitive load during reviews:

- The owner is set to the Token 2022 program, not the current program:

```
// Create the account with our program as owner
CreateAccount {
    from: self.accounts.payer,
    to: self.accounts.group,
    lamports,
    space: space as u64,
    owner: &TOKEN_2022_PROGRAM_ID,
}
```

- The payer gets _paid_ the rent during record deletion, rather than having to pay rent:

```
/// 2. `payer` - The account that will pay for the record account
```

- For token transfers, the authority must be the mint itself (the permanent delegate)

```
/// 1. The authority must be:
///    a. The mint's owner, or
```

- The argument is named `payer` now:

```
/// * `authority` - The authority account that will receive excess lamports
    or provide additional lamports
```

- The account is the destination for the lamports, not the source

```
/// 2. `payer` - The account that will pay for the record account
```

- For records, the field name is `seed` so the constant's name should be `SEED_LEN_OFFSET` and the comment should refer to `seed`

```
// Deserialize variable length data
let mut variable_data: ByteReader<'info> =
    ByteReader::new_with_offset(ctx.data, NAME_LEN_OFFSET);

// Deserialize `name`
let seed: &[u8] = variable_data.read_bytes_with_length()?;
```

- The instruction is `MintTokenizedRecord`

```
/// MintRecordToken instruction.
```

- The `tokenAccount` is `token_account` and is specifically the *associated* token account. Also `token2022` should be `token_2022_program`

```
/// 8. `tokenAccount` - The token account where we mint the record token to
/// 9. `token2022` - The Token2022 program
/// 10. `system_program` - Required for initializing our accounts
```

- It must be the record's token's owner, not the mint's owner

```
/// 1. The authority must be:
///    a. The mint's owner, or
///    b. if the class is permissioned, the authority must be the
    permissioned authority
/// 2. The record must not be frozen
pub struct TransferTokenizedRecordAccounts<'info> {
```

- The comments should describe initializing a group member, and should include those accounts instead

```
/// Initializes a Mint Close Authority.
///
///
### Accounts:
///   0. `[WRITE]`  The mint account to initialize a close authority for.
pub struct InitializeMember<'a> {
```

- The comment should describe initializing a metadata pointer

```
///
### Accounts:
///   0. `[WRITE]`  The mint account to initialize a close authority for.
pub struct InitializeMetadataPointer<'a> {
```

- The description should be about a permanent delegate

```
/// Initializes a Mint Close Authority.
///
///
### Accounts:
///   0. `[WRITE]`  The mint account to initialize a permanent delegate for.
pub struct InitializePermanentDelegate<'a> {
```

- The description should be about a transfer

```
/// Initializes a Mint Close Authority.
///
///
### Accounts:
///  0. `[writable]` The source account.
///  1. `[]` The token mint.
///  2. `[writable]` The destination account.
///  3. `[signer]` The source account's owner/delegate.
```

```
///
///
### Data:
///  0. amount (u64)
///  1. decimals (u8)
pub struct TransferChecked<'a> {
```

- The description should be about initializing a group

```
/// Initializes a Mint Close Authority.
///
///
### Accounts:
///   0. `[WRITE]`  The mint account to initialize a close authority for.
pub struct InitializeGroup<'a> {
```

- The comments reference URIs but the instruction does not support updating the URI

```
// - [8]: field (u8)
// - [9..13]: new_uri length (u32)
// - [13..13+new_uri.len()]: new_uri bytes
let mut instruction_data =
    [UNINIT_BYTE; Self::DISCRIMINATOR.len() + size_of::<u32>() +
    MAX_METADATA_LEN];

write_bytes(
    &mut instruction_data[DISCRIMINATOR_OFFSET..],
    &Self::DISCRIMINATOR,
);

// Write field at offset [8]
write_bytes(&mut instruction_data[FIELD_OFFSET..], &[3]);

// Write new_uri length at offset [9..13]
```

- The lamports are given to the payer, not the authority

```
// Can return excess lamports to authority
let lamports_diff = target_account
    .lamports()
    .saturating_sub(new_minimum_balance);
*payer.try_borrow_mut_lamports()?
    = payer.lamports().saturating_add(lamports_diff);
```

Recommendations:

Update the comments to reflect the current behavior

D3:

Zenith:

[I-3] Typos

SEVERITY: Informational	IMPACT: Informational
STATUS: Open	LIKELIHOOD: Low

Target

- program/**/*.rs

Description:

The following typos were identified:

`perfomed -> performed`

```
// 1. Update the record to be frozen since the check will be perfomed on the
    token account
```

`_EXTENSIONE_ -> _EXTENSION_`

```
const MINT_METADATA_EXTENSIONE_WITH_ADDITIONAL_METADATA: &[u8; 111] = &[
```

Recommendations:

Correct the typos

D3:

Zenith:

[I-4] `burn_tokenized_record()` should close the owner token account

SEVERITY: Informational	IMPACT: Informational
STATUS: Open	LIKELIHOOD: Low

Target

- burn_tokenized_record.rs#L93

Description:

`burn_tokenized_record()` will burn the NFT for the record and close the mint account.

However, it fails to close the token account, which is no longer useful. This means that the owner need to perform a separate TX to close the token account.

Recommendations:

Consider closing the owner token account in `burn_tokenized_record()`.

D3:

Zenith:

[I-5] `burn_tokenized_record()` might not refund rent to the original payer

SEVERITY: Informational	IMPACT: Informational
STATUS: Open	LIKELIHOOD: Low

Target

- burn_tokenized_record.rs#L121

Description:

`burn_tokenized_record()` will burn close the mint account and refund the rent to the `payer`.

However, the `payer` might not be the same payer who funded the rent initially, as it could be the previous owner or the delegate.

Recommendations:

Consider storing the rent refund in the program and add a new instruction for the original payer to claim the rent.

D3:

Zenith:

[I-6] unnecessary refund code in `DeleteRecord::execute()`

SEVERITY: Informational	IMPACT: Informational
STATUS: Open	LIKELIHOOD: Low

Target

- delete_record.rs#L77-L85

Description:

`DeleteRecord::execute()` calls `Record::delete_record_unchecked()` and perform a `resize_account()`, which will handle the lamport refunds.

However, after that it attempts to refund excess lamport in the record again, without updating the payer's lamports.

But this is not necessary and can be removed since `resize_account()` already handled it.

```
impl<'info> DeleteRecord<'info> {
    ..
    pub fn execute(&self) -> ProgramResult {
        // Safety: The account has already been validated
        unsafe {
            Record::delete_record_unchecked(self.accounts.record,
    self.accounts.payer)?;

            // Refund the payer of all the lamports
            self.accounts
                .payer
                .borrow_mut_lamports_unchecked()

    .checked_add(*self.accounts.record.borrow_lamports_unchecked(
                    ))
                .and_then(|x| x.checked_sub(ONE_LAMPORT_RENT))
                .ok_or(ProgramError::InvalidAccountData)?;

            *self.accounts.record.borrow_mut_lamports_unchecked() = ONE_
                LAMPORT_RENT;
```

```
        }

        Ok(())
    }
}
```

D3:

Zenith:

[I-7] Semantics of `Mint::check_initialized()` are unclear

SEVERITY: Informational	IMPACT: Informational
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/token2022/state.rs

Description:

There are separate functions for checking a `Mint`'s discriminator and for checking its initialization state but they both only check for the present of the discriminator:

```
    /// # Safety
    /// Token Program ID is not checked
    pub unsafe fn check_discriminator_unchecked(data: &[u8]) -> Result<(),
    ProgramError> {
        if
    data[TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET].ne(&MINT_DISCRIMINATOR) {
            return Err(ProgramError::InvalidAccountData);
        }

        Ok(())
    }

    pub fn check_initialized(account_info: &AccountInfo) -> Result<bool,
    ProgramError> {
        ...

        if
    data[TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET].ne(&MINT_DISCRIMINATOR) {
            return Ok(false);
        }

        Ok(true)
    }
}
```

While it's true that the token2022 program *currently* only writes the discriminator *after* the

account is initialized, it's not technically true that the account is considered as initialized, since there is an actual `is_initialized` field that example code checks

Recommendations:

Check the `is_initialized` field separately from the discriminator

D3:

Zenith:

[I-8] CU optimizations

SEVERITY: Informational	IMPACT: Low
STATUS: Open	LIKELIHOOD: Low

Target

- program/src/instructions/mint_tokenized_record.rs
- program/src/instructions/mint_tokenized_record.rs
- program/src/state/class.rs
- program/src/state/record.rs
- program/src/state/record.rs

Description:

The token group extension is also a static extension and its space can be pre-allocated as well:

```
// Space of all our static extensions
let space = TOKEN_2022_MINT_LEN + TOKEN_2022_MINT_BASE_LEN +
    TOKEN_2022_GROUP_POINTER_LEN;

let lamports = Rent::get()?.minimum_balance(space + TOKEN_2022_GROUP_LEN);
```

The token member extension is also a static extension. It can be pre-allocated if its initialization is done prior to the setting of the metadata

```
// To avoid resizing the mint, we calculate the correct lamports for our
    token AOT with:
// 1. `space` - The sum of the above static extension lengths
// 2. `metadata_data.len()` - The full length of the metadata data
// 3. `TOKEN_2022_MEMBER_LEN` - The length of the member extension
let lamports = Rent::get()?.minimum_balance(
    space
        + unsafe {

    Record::get_metadata_len_unchecked(&self.accounts.record.try_borrow_data()?)?
        }
        + TOKEN_2022_MEMBER_LEN,
```

```
);
```

The resize never needs to set `zero_out` to true, since the instruction never increases the account's size after decreasing it

```
if new_len ≠ current_len {
    resize_account(class, payer, new_len, new_len < current_len)?;
}
```

The same is true here, so the `update_data_unchecked()` function could take in a new argument that's always passed `true` and used in `resize_account()`

```
resize_account(record, authority, new_len, new_len < current_len)?;
```

The same is true for delete:

```
resize_account(record, payer, 1, true)?;
```

Recommendations:

Apply the recommendations above

D3:

Zenith: